

As a Sr Lead - QA working in Insurance Domain create a Test Strategy Document. Ask me 5 questions one by one for creating it, Ask me the next question after i provide the answer to the previous question, Ensure that the Test Strategy Document is as per industry standard and follows Agile Process

Great — I'll build this with you step by step so it reflects your real environment and aligns with Agile best practices.

Question 1 of 5

What is the **scope of the application/system** for which you want the Test Strategy?

Please include details like:

- Type of system (e.g., Policy Admin, Claims, Underwriting, Billing, Digital Portal, APIs, etc.)
- Platforms (Web, Mobile, APIs, Batch, etc.)
- Any integrations (e.g., third-party vendors, payment gateways, core insurance systems)

Once you answer, I'll move to the next question.

This is a Policy Admin system built for Web, Mobile, API

Great, that gives a solid starting point.

Question 2 of 5

What is the **delivery model and Agile setup** for this project?

Please specify:

- Agile framework (e.g., Scrum, SAFe, Kanban, hybrid)
- Sprint duration
- Team structure (e.g., QA in-sprint, separate QA team, POD-based model)
- Release frequency (e.g., every sprint, monthly, quarterly)

Once you share this, I'll move to the next question.

Scrum Framework with Sprint Duration 2 Weeks, QA in Sprint structure, and Monthly Release

Perfect — that's a well-structured Agile setup.

Question 3 of 5

What are the **testing types and levels** you want to include in the strategy?

For example:

- Functional (UI, API)
- Integration testing
- Regression testing
- UAT support
- Performance / Load testing
- Security testing
- Automation (UI/API)
- Any domain-specific testing (e.g., policy lifecycle validations, regulatory compliance)

Feel free to list everything you expect QA to cover. Once you respond, I'll move to the next question.

All of the above

Great — we'll include a comprehensive, end-to-end testing scope.

Question 4 of 5

What is your **test environment and data strategy**?

Please provide details like:

- Number and types of environments (e.g., Dev, QA, UAT, Staging)
- Test data approach (masked production data, synthetic data, on-demand creation)
- Any dependencies or constraints (e.g., shared environments, data refresh cycles, external system availability)

Once you share this, I'll ask the final question.

DEV/QA/UAT/Staging Env, On Demand Creation and masked Prod Data with Data Refresh Cycles every 2 months

Excellent — that's a mature setup.

Question 5 of 5 (Final)

What tools and technologies are used across the QA lifecycle?

Please include:

- Test management tools (e.g., JIRA, Zephyr, TestRail)
- Automation tools (UI/API)
- CI/CD tools
- Performance testing tools
- Defect tracking tools (if different)
- Any reporting/dashboard tools

Once you provide this, I'll create a complete **industry-standard Agile Test Strategy Document** tailored to your setup.

ADO and Playwright

Great — thanks for all the inputs. Based on your responses, here is a **comprehensive, industry-standard Agile Test Strategy Document** tailored to your role as a Sr. Lead QA in the Insurance domain.

Test Strategy Document

Project: Policy Administration System

Domain: Insurance

Prepared By: Sr. Lead QA

Methodology: Agile (Scrum)

1. Introduction

This document outlines the overall **test strategy, approach, and governance** for the Policy Administration System covering Web, Mobile, and API layers. The objective is to ensure high-quality, scalable, and compliant releases aligned with business and regulatory requirements in the insurance domain.

2. Scope of Testing

In Scope

- Policy lifecycle: Quote → Bind → Issue → Endorsement → Renewal → Cancellation
- Platforms:
 - Web application
 - Mobile application

- APIs (internal & external integrations)
- Integration with third-party systems (if applicable)

Out of Scope

- Legacy systems not part of current release scope
 - Third-party systems (limited to integration validation)
-

3. Agile Delivery Model

- Framework: Scrum
- Sprint Duration: 2 Weeks
- QA Model: In-Sprint QA (Shift-left approach)
- Release Cycle: Monthly

QA Involvement

- Participate in backlog grooming & sprint planning
 - Define acceptance criteria & test scenarios early
 - Continuous testing within sprint
 - Support sprint demos and retrospectives
-

4. Test Levels & Types

4.1 Test Levels

- Unit Testing (by Dev team)
- Integration Testing
- System Testing
- User Acceptance Testing (UAT support)

4.2 Test Types

Functional Testing

- UI Testing (Web & Mobile)
- API Testing
- Business workflow validation (Policy lifecycle)

Non-Functional Testing

- Performance & Load Testing
- Security Testing
- Usability Testing

Regression Testing

- Automated regression suite using Playwright
- Executed in every sprint and before monthly release

Automation Testing

- UI Automation (Web)
- API Automation
- CI/CD integrated test execution

Domain-Specific Testing

- Insurance rules validation (premium calculation, underwriting rules)
 - Regulatory compliance validation
-

5. Test Environment Strategy

Environments

- DEV – Development & unit testing
- QA – Functional & integration testing
- UAT – Business validation
- Staging – Pre-production validation

Environment Management

- Shared environments with controlled access
 - Deployment aligned with sprint cadence
-

6. Test Data Management

- Masked production data for realistic scenarios
- On-demand test data creation for specific cases

- Data refresh cycle: Every 2 months

Challenges & Mitigation

- Data dependency → Use synthetic data generation
 - Data inconsistency → Maintain baseline datasets
-

7. Test Management & Defect Tracking

- Test Management Tool: Azure DevOps
- Defect Tracking: Azure DevOps

Practices

- Test cases linked to user stories
 - Defects tracked with severity & priority
 - Traceability matrix maintained
-

8. Automation Strategy

Approach

- Shift-left automation
- Automate critical business flows first
- Maintain reusable and scalable framework

Tools

- UI & API Automation: Playwright

Execution

- Integrated with CI/CD pipelines
 - Triggered on code check-in and nightly runs
-

9. CI/CD Integration

- Continuous Integration with automated test execution
- Build validation through smoke tests
- Regression suite execution before release

10. Entry & Exit Criteria

Entry Criteria

- User stories defined with acceptance criteria
- Test environment ready
- Test data available

Exit Criteria

- All planned test cases executed
 - Critical & high defects closed
 - Regression suite passed
 - UAT sign-off obtained
-

11. Defect Management

Defect Lifecycle

New → Assigned → In Progress → Fixed → Retest → Closed / Reopened

Severity Levels

- Critical: System blocking
 - High: Major functionality impacted
 - Medium: Partial impact
 - Low: Minor issues
-

12. Risk Management

Risk	Impact	Mitigation
Environment instability	High	Environment monitoring & backup plan
Data dependency	Medium	Synthetic data strategy
Tight sprint timelines	High	Early QA involvement (shift-left)
Integration failures	High	Mock services & stubs

13. Reporting & Metrics

Key Metrics

- Test execution progress
- Defect density & leakage
- Automation coverage
- Pass/Fail trends
- Sprint QA burndown

Reporting

- Daily status updates
- Sprint QA summary
- Release quality report

14. Roles & Responsibilities

Role	Responsibility
QA Lead	Strategy, planning, governance
QA Engineers	Test design, execution, automation
Developers	Unit testing, defect fixes
Product Owner	Acceptance criteria, UAT
Scrum Master	Process facilitation

15. Continuous Improvement

- Retrospective-driven improvements
 - Automation optimization
 - Process refinement based on metrics
 - Adoption of best practices in Agile QA
-

16. Compliance & Governance

- Ensure adherence to insurance regulations
 - Audit-ready test artifacts
 - Data privacy (masked production data usage)
-

If you want, I can next:

- Convert this into a **Word document format with headings/styles**
- Tailor it for **specific insurance lines (Life, Health, P&C)**
- Add **RACI matrix or detailed KPI dashboard**